TECHNICAL REFERENCE & CURRICULUM GUIDE

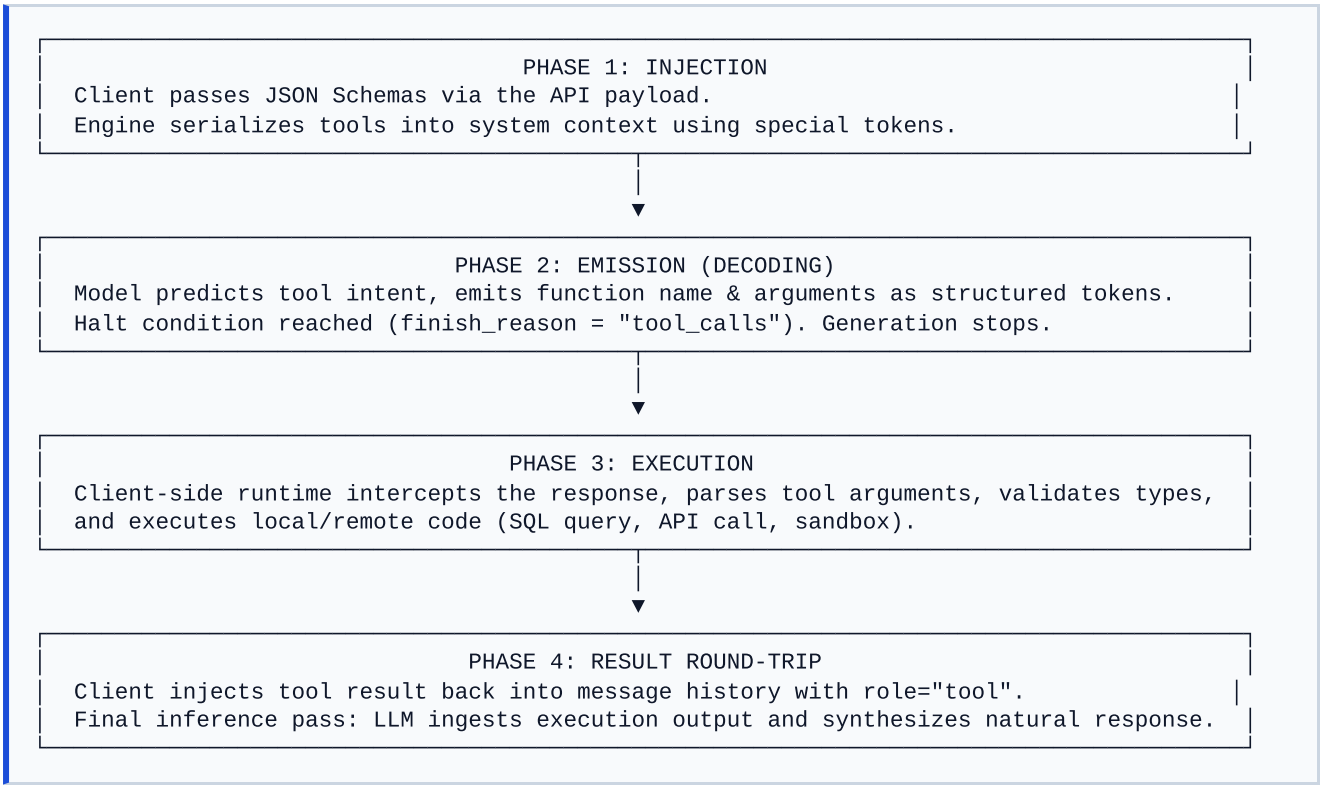
# Module 8 (Part 1): Tool Calling & Agentic Workflows

*Module 8 Part 1 deconstructs the computational mechanics, schema design standards, cross-provider protocols, and orchestration loops that power modern tool-augmented LLMs and agentic pipelines.*

---

## 8.1 Tool / Function Calling Under the Hood: The Four-Phase Lifecycle

Large Language Models do not possess execution environments or live network interfaces. An LLM cannot natively query a database, make an HTTP request, or run shell scripts. **Tool calling (or Function Calling)** is an architectural pattern that turns an LLM into a structured query planner while offloading execution to the client runtime.




### Phase 1: Schema Injection (Prompt Serialization)

When tools are declared in an API payload, the serving engine converts the JSON Schema declarations into a structured system prompt prefix using the model's specialized template.

- In open-weight and proprietary models (such as ChatML, LLaMA-3.1/3.3, or Mistral), tool schemas are wrapped between dedicated sentinel tokens:

```
<|start_header_id|>system<|end_header_id|>
You have access to the following tools:
{"type": "function", "function": {"name": "get_stock_price", ...}}
To call a tool, respond with a JSON object matching the schema.<|eot_id|>
```

- These schemas enter the model's standard self-attention mechanism. Every parameter description, type constraint, and enum acts as a conditioning signal across the model's attention heads.

### Phase 2: Model-Emitted Call (Constrained Token Generation)

When a user prompt requires external data or action, the model attends over the available function definitions:

1. Instead of sampling general conversational tokens, the model's logits favor a specialized control token (e.g., `<|start_tool_call|>` or OpenAI's internal tool flag).
2. The model autoregressively generates the tool name and a stringified JSON object containing the function arguments.
3. Once the closing bracket `}` is emitted, the model emits an end-of-call sentinel token.
4. The server returns HTTP 200 with `finish_reason: "tool_calls"` (OpenAI) or `stop_reason: "tool_use"` (Anthropic), pausing generation.

### Phase 3: Client-Side Execution (The Isolation Boundary)

The inference server yields control back to the client application:

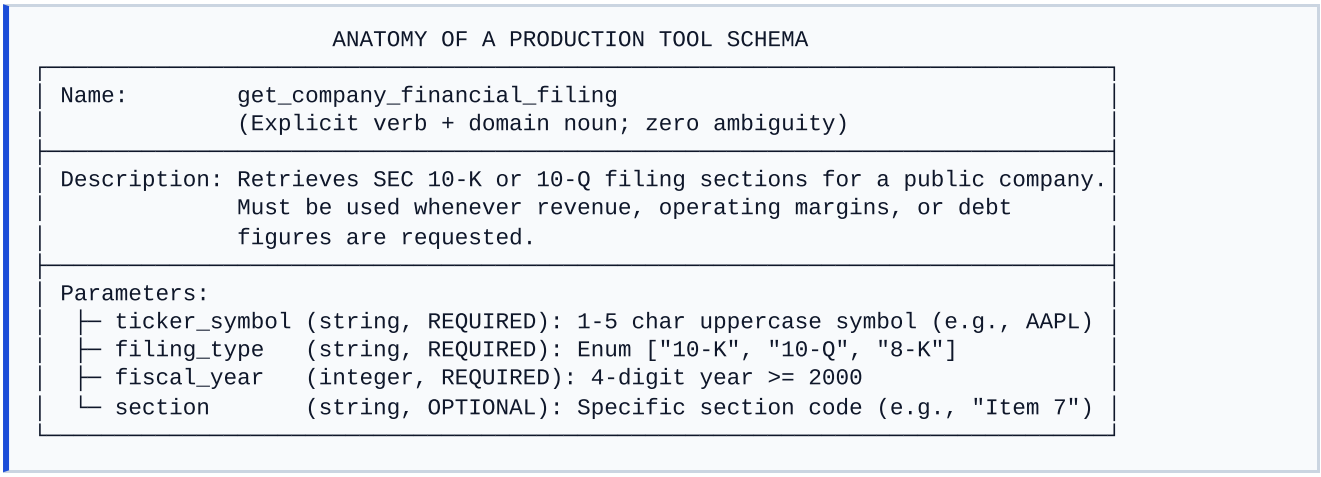
1. The client intercepts the structured response payload.
2. It validates the emitted arguments against runtime data models (e.g., Pydantic).
3. The client executes the matching code block locally or remotely (e.g., executing a `SELECT` statement in PostgreSQL, dispatching a REST call to Stripe, or running Python code in an isolated Docker container).
4. **Security Principle:** The LLM never touches production infrastructure directly; all authentication, parameter sanitization, and database permissions remain isolated within the client's execution environment.

### Phase 4: Result Round-Trip & Final Synthesis

1. The client serializes the execution result into a string (typically JSON) and appends it to the conversation history.
2. The message is tagged with the specific role ( `role="tool"` in OpenAI/Gemini; `role="user"` containing a `tool_result` content block in Anthropic) and paired with the original `tool_call_id`.
3. The client submits the updated message history back to the model.
4. The LLM reads the tool output during its prefill phase and autoregressively generates a grounded, natural-language response (or emits subsequent tool calls if more data is needed).

## 8.2 Designing Robust Tool Schemas

The reliability of tool calling depends directly on schema expressiveness. Ambiguous function names, missing docstrings, or loose parameter typing cause hallucinated arguments, missing required values, and incorrect tool selection.




### 1. Function Naming Conventions

- **Use explicit `<verb>_<domain_entity>` formats:** `search_knowledge_base`, `fetch_customer_order`, `calculate_compound_interest`.
- **Avoid generic, overloaded verbs:** Names like `run()`, `process()`, `do_query()`, or `handle_data()` provide no semantic prior to the router attention heads, increasing tool misrouting.
- **Keep names distinct:** Having both `get_user_info` and `fetch_user_data` in the same toolset creates logit competition where attention splits across near-identical functions.

### 2. Description Engineering (The Semantic Guide)

The description is the primary mechanism the LLM uses to decide *whether* and *when* to invoke a tool.

- **State the explicit trigger condition:** Clarify what user intents map to this tool.
- **State negative routing boundaries:** Explicitly define when **not** to call the tool (*"Do NOT use this tool for private companies; use search_crunchbase instead"*).
- **Format requirements:** Clarify any implicit transformations the model should apply before generating arguments (*"Convert all dates to ISO-8601 YYYY-MM-DD format before passing"*).

### 3. Parameter Typing & Structural Constraints

Use standard JSON Schema specifications (Draft 7 / 2020-12):

- **Primitive Types:** `string`, `number`, `integer`, `boolean`, `array`, `object`.
- **Enums:** Constrain categorical inputs strictly (`"enum": ["asc", "desc"]`). Never rely on open text strings for fixed options.
- **Numeric Boundaries:** Use `"minimum"` and `"maximum"` to prevent impossible ranges (e.g., `"minimum": 1, "maximum": 100` for page size).
- **Regex Patterns:** Enforce structural strings with `"pattern": "^[A-Z]{3,5}$"` for ticker symbols or ISO currency codes.

#### 4. Required vs. Optional Discipline

- **Required Parameters:** Every field strictly necessary for backend execution must be marked in the `"required": [...]` array.
- **Optional Parameters:** If a parameter is optional, specify its **default fallback behavior** in its description (*"If omitted, defaults to 10 records"*).
- **Avoid Boolean Overload:** Instead of optional boolean flags like `is_summary: true`, use explicit string enums like `"detail_level": ["summary", "detailed", "raw"]`, which provide better semantic grounding.

## 8.3 Single Tool Call End-to-End: OpenAI Function Calling

The following implementation demonstrates the complete lifecycle using the official OpenAI Python SDK (v1.x+): registering a typed tool via Pydantic, executing the model-emitted call, injecting the result, and retrieving the final response.

```
import json
from typing import Literal
from pydantic import BaseModel, Field
from openai import OpenAI

client = OpenAI()

# ---------------------------------------------------------------------------
# 1. Define Tool Schema via Pydantic & JSON Schema Converter
# ---------------------------------------------------------------------------
class DatabaseQueryInput(BaseModel):
    table: Literal["users", "subscriptions", "invoices"] = Field(
        description="The target relational table to query."
    )
    user_id: int = Field(
        description="The unique integer ID of the customer."
    )
    status_filter: Literal["active", "canceled", "past_due"] = Field(
        default="active",
        description="Filter records by operational status."
    )

tools = [
    {
        "type": "function",
        "function": {
            "name": "query_customer_database",
            "description": "Look up active subscriptions and billing status for a customer
by user_id.",
            "parameters": {
                "type": "object",
                "properties": {
                    "table": {
                        "type": "string",
                        "enum": ["users", "subscriptions", "invoices"],
                        "description": "The target relational table to query."
                    },
                    "user_id": {
                        "type": "integer",
                        "description": "The unique integer ID of the customer."
                    },
                    "status_filter": {
                        "type": "string",
                        "enum": ["active", "canceled", "past_due"],
                        "description": "Filter records by operational status."
                    }
                },
                "required": ["table", "user_id"],
                "additionalProperties": False
            }
        }
    }
]

# ---------------------------------------------------------------------------
# 2. Local Mock Execution Function (The Sandbox/DB Implementation)
# ---------------------------------------------------------------------------
def query_customer_database(table: str, user_id: int, status_filter: str = "active") -> str:
    # In production: connect to PostgreSQL/MySQL via connection pool
    mock_db = {
        ( "subscriptions", 1042 ): {
            "subscription_id": "sub_8832a",
            "plan": "Enterprise Pro Tier",
            "monthly_usd": 499.00,
            "status": "active",
            "renewal_date": "2026-12-01"
```

## 8.4 Cross-Provider Protocols: OpenAI vs. Claude vs. Gemini

Each frontier provider utilizes a different message schema, wire structure, and payload specification for tool calling. Understanding these architectural differences is essential for multi-provider gateways and abstraction layers.

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              WIRE FORMAT COMPARISON                          │
│                                                                              │
│  OpenAI:                                                                     │
│    tools=[{"type": "function", "function": {"name": ..., "parameters": {...}}}]  │
│    Model emits: choices[0].message.tool_calls = [{"id": ..., "function": ...}]   │
│    Return role: "tool" with "tool_call_id"                                   │
│                                                                              │
│  Anthropic Claude:                                                           │
│    tools=[{"name": ..., "description": ..., "input_schema": {...}}]          │
│    Model emits: content=[{"type": "tool_use", "id": ..., "name": ..., "input": {...}}]  │
│    Return role: "user" with content=[{"type": "tool_result", "tool_use_id": ...}]      │
│                                                                              │
│  Google Gemini:                                                              │
│    tools=[{"function_declarations": [{"name": ..., "parameters": {...}}]}]   │
│    Model emits: parts[0].function_call = {"name": ..., "args": {...}}       │
│    Return role: "user" with parts=[{"function_response": {"name": ..., "response":...}}]  │
└──────────────────────────────────────────────────────────────────────────────┘
```

### 1. Architectural & API Matrix

| Feature | OpenAI (v1 API) | Anthropic (Claude 3.5 / 4 Messages API) | Google Gemini (v1beta API) |
|---|---|---|---|
| **Schema Field Name** | `"parameters"` | `"input_schema"` | `"parameters"` inside `function_declarations` |
| **Tool Definition Wrapper** | `{"type": "function", "function": {...}}` | `{"name": ..., "input_schema": {...}}` | `{"function_declarations": [{...}]}` |
| **Model Emission Format** | `message.tool_calls[i]` (Dedicated array) | Interleaved `content` block (`type="tool_use"`) | `parts[i].function_call` |
| **Output Arguments Data Type** | Stringified JSON (`"{\"user_id\": 1042}"`) | Native parsed dictionary (`{"user_id": 1042}`) | Native parsed structure / proto map |
| **Call Tracking ID** | Required (`call_abc123`) | Required (`toolu_01...`) | Implicit / Function Name matching |
| **Tool Response Role** | Dedicated `"tool"` role | Standard `"user"` role with `tool_result` block | Standard `"user"` role with `function_response` part |
| **Stop Signal** | `finish_reason: "tool_calls"` | `stop_reason: "tool_use"` | `finish_reason: "STOP"` |

## 2. Anthropic Claude Tool Use (Python Implementation)

```
import anthropic

client = anthropic.Anthropic()

claude_tools = [
    {
        "name": "lookup_weather",
        "description": "Fetch current temperature and conditions for a city.",
        "input_schema": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "City name, e.g. London"},
                "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
            },
            "required": ["city"]
        }
    }
]

# Initial turn
response = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1024,
    tools=claude_tools,
    messages=[{"role": "user", "content": "What is the temperature in Tokyo right now?"}]
)

# Anthropic uses content blocks for text and tool use
if response.stop_reason == "tool_use":
    tool_use_block = next(block for block in response.content if block.type == "tool_use")
    tool_id = tool_use_block.id
    tool_name = tool_use_block.name
    tool_input = tool_use_block.input  # Already a native Python dict!

    # Execute mock action
    mock_weather_output = "{\"temperature\": 18, \"condition\": \"Clear\", \"unit\":
\"celsius\"}"

    # Send tool result back under role="user" with a tool_result content block
    follow_up = client.messages.create(
        model="claude-3-5-sonnet-20241022",
        max_tokens=1024,
        tools=claude_tools,
        messages=[
            {"role": "user", "content": "What is the temperature in Tokyo right now?"},
            {"role": "assistant", "content": response.content},
            {
                "role": "user",
                "content": [
                    {
                        "type": "tool_result",
                        "tool_use_id": tool_id,
                        "content": mock_weather_output
                    }
                ]
            }
        ]
    )
    print(follow_up.content[0].text)
```

### 3. Google Gemini Function Calling (Python Google GenAI SDK)

```
from google import genai
from google.genai import types

client = genai.Client()

def get_current_stock_quote(symbol: str) -> dict:
    """Fetch real-time stock price and market status for a given ticker."""
    return {"symbol": symbol, "price": 182.45, "currency": "USD", "change_percent": "+1.2%"}

# Gemini automatically inspects Python type hints and docstrings
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="What is the current stock price of GOOG?",
    config=types.GenerateContentConfig(
        tools=[get_current_stock_quote]  # Direct callable registration
    )
)

# Check for function call
if response.function_calls:
    fn_call = response.function_calls[0]
    print(f"Gemini requested tool: {fn_call.name} with args: {fn_call.args}")

    # Execute tool
    tool_output = get_current_stock_quote(**fn_call.args)

    # Return output back using function_response part
    final_response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=[
            types.Content(role="user", parts=[types.Part.from_text(text="What is the current
stock price of GOOG?")]),
            response.candidates[0].content,
            types.Content(
                role="user",
                parts=[
                    types.Part.from_function_response(
                        name=fn_call.name,
                        response={"result": tool_output}
                    )
                ]
            )
        ]
    )
    print(final_response.text)
```

## 8.5 Parallel Tool Calling & Tool Choice Strategies

In complex user prompts, an LLM often requires data from multiple independent endpoints simultaneously. Modern engines support **Parallel Tool Calling**, allowing the model to emit multiple tool calls in a single autoregressive generation step.

```
                    [USER PROMPT]
 "Compare the stock prices of Apple, Microsoft, and Nvidia."
                            |
                            ▼
           [MODEL SINGLE GENERATION PASS]
Emits 3 independent tool calls in a single response:
├─ Call 1: get_stock_price(ticker="AAPL") [id: call_01]
├─ Call 2: get_stock_price(ticker="MSFT") [id: call_02]
└─ Call 3: get_stock_price(ticker="NVDA") [id: call_03]
                            |
                            ▼
       [CLIENT CONCURRENT EXECUTION LAYER]
asyncio.gather(call_01, call_02, call_03)
All 3 network requests execute simultaneously in ~150ms
                            |
                            ▼
       [INJECT ALL 3 RESULTS SIMULTANEOUSLY]
messages.append(tool_result_01)
messages.append(tool_result_02)
messages.append(tool_result_03)
                            |
                            ▼
             [FINAL SYNTHESIS PASS]
```

### 1. Parallel Tool Calling Mechanics

- **Sequential Latency Trap:** Without parallel calling, retrieving 3 independent APIs sequentially requires 3 separate inference round-trips:

$$\text{Total Time} = 3 \times (\text{LLM_Latency} + \text{API_Latency}) \approx 3 \times (800\text{ms} + 200\text{ms}) = 3{,}000\text{ms}$$

- **Parallel Execution Speedup:** The model generates an array of tool calls in one forward pass. The client executes all network requests concurrently using asynchronous I/O ( `asyncio.gather` ), reducing the wall-clock network time to max(latency):

$$\text{Total Time} = \text{LLM_Latency}_1 + \max(\text{API}_1, \text{API}_2, \text{API}_3) + \text{LLM_Latency}_2 \approx 800\text{ms} + 220\text{ms} + 600\text{ms} = 1{,}620\text{ms}$$

### 2. Tool-Choice Strategies ( `tool_choice` )

The `tool_choice` parameter governs how aggressively the model's logits are steered toward calling tools versus outputting plain conversational text.

```
# Strategy 1: "auto" (The Default Dynamic Mode)
# The model decides dynamically whether to speak or call any tool.
payload = {"tool_choice": "auto"}

# Strategy 2: "none" (Forced Suppression)
# The model is explicitly forbidden from calling any tools.
# Useful for final clarification passes or fallback formatting.
payload = {"tool_choice": "none"}

# Strategy 3: "required" (Forced Tool Activation)
# The model MUST call at least one tool from the registered tools array.
# It cannot emit a direct text response to the user.
payload = {"tool_choice": "required"}

# Strategy 4: Named Tool Constraint (Forced Specific Tool)
# Constrains the model's output specifically to one designated tool schema.
payload = {
    "tool_choice": {
        "type": "function",
        "function": {"name": "query_customer_database"}
    }
}
```

### 3. Production Asynchronous Parallel Execution (Python)

```
import asyncio
import json
from openai import AsyncOpenAI

async_client = AsyncOpenAI()

async def mock_fetch_price(ticker: str) -> dict:
    await asyncio.sleep(0.2)  # Simulate non-blocking network I/O
    prices = {"AAPL": 224.23, "MSFT": 428.15, "NVDA": 138.25}
    return {"ticker": ticker, "price": prices.get(ticker, 0.0)}

async def handle_parallel_tool_calls(user_query: str):
    tools = [{
        "type": "function",
        "function": {
            "name": "mock_fetch_price",
            "description": "Get current price for a ticker.",
            "parameters": {
                "type": "object",
                "properties": {"ticker": {"type": "string"}},
                "required": ["ticker"]
            }
        }
    }]

    messages = [{"role": "user", "content": user_query}]

    # Pass 1: Model identifies multiple entities and emits parallel calls
    response = await async_client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        tools=tools,
        parallel_tool_calls=True  # Enabled by default on modern checkpoints
    )

    msg = response.choices[0].message
    if not msg.tool_calls:
        return msg.content

    messages.append(msg)

    # Coroutine execution tasks
    async def run_tool(call):
        args = json.loads(call.function.arguments)
        res = await mock_fetch_price(ticker=args["ticker"])
        return {
            "role": "tool",
            "tool_call_id": call.id,
            "content": json.dumps(res)
        }

    # Execute all emitted tool calls concurrently
    tool_results = await asyncio.gather(*(run_tool(call) for call in msg.tool_calls))

    # Append all results back into message history
    messages.extend(tool_results)

    # Pass 2: Final synthesis
    final_res = await async_client.chat.completions.create(
        model="gpt-4o",
        messages=messages
    )
    return final_res.choices[0].message.content
```

## 8.6 Multi-Step Tool Chains: Sequential Dependencies

In real-world agentic workflows, problems cannot always be resolved in a single parallel step. A task often requires **sequential dependent execution**, where the output of Tool A forms the required input parameters for Tool B.




### 1. State Invariants in Multi-Step Chains

- **Context Preservation:** Every intermediate `assistant` tool-call request and `tool` observation must remain in the message history array. If an intermediate step is dropped, the LLM loses its operational trace, triggering loops or repeated calls.
- **Context Budget Degradation:** Long sequential chains rapidly consume context window tokens ($O(T^2)$ growth over turns). Large API payloads (e.g., retrieving a 2,000-line JSON payload in Step 1) must be minified or summarized before injection.
- **Defensive Termination Limits:** Without strict guardrails, an agent in a multi-step loop can enter an **infinite execution cycle** if an API returns an unhandled validation error. Production systems must enforce an explicit `max_iterations` counter (typically 5 to 10 iterations max).

## 2. Production Multi-Step Orchestration Engine (Python)

```
import json
from typing import Dict, Any, List
from openai import OpenAI

client = OpenAI()

# ---------------------------------------------------------------------------
# Mock Environment APIs with Sequential Dependencies
# ---------------------------------------------------------------------------
def search_contacts(query: str) -> str:
    contacts = {"Bob": {"user_id": "usr_9918", "email": "bob@example.com"}}
    return json.dumps(contacts.get(query, {"error": "Contact not found"}))

def get_account_balance(account_type: str) -> str:
    balances = {"checking": 450.00, "savings": 1200.00}
    return json.dumps({"account_type": account_type, "balance": balances.get(account_type,
0.0)})

def execute_transfer(recipient_id: str, amount: float) -> str:
    # Mutating operation
    return json.dumps({"status": "SUCCESS", "transfer_id": "tx_441829", "amount": amount,
"to": recipient_id})

TOOL_MAP = {
    "search_contacts": search_contacts,
    "get_account_balance": get_account_balance,
    "execute_transfer": execute_transfer
}

AGENT_TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "search_contacts",
            "description": "Look up a user ID by person's name.",
            "parameters": {
                "type": "object",
                "properties": {"query": {"type": "string"}},
                "required": ["query"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "get_account_balance",
            "description": "Check current funds in an account.",
            "parameters": {
                "type": "object",
                "properties": {"account_type": {"type": "string", "enum": ["checking",
"savings"]}},
                "required": ["account_type"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "execute_transfer",
            "description": "Transfer funds to a verified recipient ID.",
            "parameters": {
                "type": "object",
                "properties": {
```

## Technical Summary of Module 8 Part 1

| Component | Technical Invariant | Primary Failure Mode | Mitigation Architecture |
|---|---|---|---|
| **8.1 Lifecycle** | 4-phase round trip: Injection → Emission → Execution → Round-Trip | Model hallucinates execution output without calling tool | Constrain logits via `tool_choice: "required"` or parse `finish_reason` strictly |
| **8.2 Schemas** | JSON Schema standard (Draft 7 / 2020-12) | Ambiguous function names cause router misfires | Explicit `<verb>_<entity>` naming, enum bounds, docstring rules |
| **8.3 OpenAI API** | Explicit `role="tool"` with matching `tool_call_id` | Missing `tool_call_id` throws HTTP 400 Bad Request | Strictly append assistant tool request followed by tool outputs |
| **8.4 Cross-Provider** | Divergent wire specifications (OpenAI vs. Claude vs. Gemini) | Type mismatches and argument parsing crashes across APIs | Provider abstraction wrappers; native dict handling for Claude |
| **8.5 Parallel Calls** | Single-pass multi-emission executed via `asyncio.gather` | Sequential bottlenecking multiplies latency $N\times$ | Concurrent client execution; independent parameter dependency check |
| **8.6 Multi-Step Chains** | Cyclic state loop with monotonic history growth | Infinite loops, out-of-context token overflows | Enforce `max_iterations` ceiling; minify intermediate tool JSON |